

Using the lightpen

This document describes the lightpen simulation implemented in the emulator.

Implementation

Pseudo-lightpen support is implemented in the pdp1 emulator in pdp1.c.

While it can't exactly duplicate the function of the original hardware lightpen because there isn't one, it can very closely simulate it.

It works by reading commands from the same file descriptor used by the *dpy* instruction to send display commands to a display client.

The reads are non-blocking. Each command is an unsigned 32 bit value, `uint32_t`, although any 32 bit value will work. The command has the format:

`0xFFc p p p p p`

The high byte is always FF to signal a lightpen command.

The 4 bit c value is either 1 to indicate the lightpen was raised or 0 to indicate it is on the screen.

The 5 bytes of p are the packed x and y coordinates.

The coordinates consist of two 10 bit 1's complement values with 0 meaning the center of the screen, the same as the coordinates used with the *dpy* instruction.

They are packed:

$(x \ll 10) \mid y$

When a *dpy* instruction completes 35 microseconds after it is issued, any queued commands are read and only the last one processed. This aligns the lightpen coordinates with the instruction.

If the coordinates of the point displayed match the last lightpen coordinates within the aperture setting and the lightpen is on the screen, then a hit has occurred.

When a hit occurs, program flag 3 is set and bit 0 in the status returned by *cks* is set to 1.

The *cks* status is reset by the next executed *dpy* command. The program flag is not automatically reset.

IMPORTANT

The lightpen status is only checked when completion is requested. If there is no completion event, no update will occur. This matches the behavior of the original hardware.

This means that *dpy-i*, no completion requested, will not see any lightpen events. However, the form that requests completion, *dpy-i 4000* or in am1, *dpy C* followed by an *ioh* will work.

The aperture

The original lightpen had a set of 6 aperture attachments to control the field of view of the lightpen, ranging from an opening of 0.05" to 0.30" in 0.05" increments. These corresponded to the pixels that would count as a hit in its view.

The simulated lightpen pixel equivalents are:

Aperture	Radius in pixels
0.50	3
0.10	6
0.15	8
0.20	11
0.25	14
0.30	17

Am1 has an include file that defines these, `#include <LIGHTPEN/lightpen.ah>`.

Enabling the lightpen and setting the aperture

An extended *dpy* command has been provided, *dpy 3000, 723000*.

It expects the IO register to contain the aperture size to use in bits 8-17.

Bit 9 enables or disables the lightpen, 1 to enable, 0 to disable.

For example:

```
lio 001006
dpy 3000
```

enables the lightpen with an aperture radius of 6 pixels, a circle with a diameter of 12 pixels on the screen.

This command does not support completion, wait is ignored. Thus *dpy* and *dpy-i* operate identically.

What uses it?

The p7sim Type 30 display emulator has support for this.

It must be started with a *-l* command line flag to enable it.

When enabled, the lightpen is simulated by the mouse.

As long as the left button is pressed, the cursor is the lightpen. Any pixel being drawn that is within the aperture around the cursor will be a hit.

There is also support in the alternal Type 30 implementation found at <https://github.com/lsysxp/PDP1-DPY.git>.